

CRUD Operations Using PDO

Learning Goals

- Implement the complete CRUD cycle in PHP using PDO's `prepare` and `execute` pattern
- Capture auto-incremented IDs with `lastInsertId()` for targeted record operations
- Use `fetchAll()` for multiple records and `fetch()` for single-record retrieval
- Safely update and delete specific records using parameterized `WHERE` clauses

The Goal and the Tools

CRUD is the goal. PHP, PDO, and MySQL are the tools.

Goal	SQL	PHP/PDO pattern
Create	INSERT	prepare → execute
Read	SELECT	prepare → execute → fetch
Update	UPDATE	prepare → execute
Delete	DELETE	prepare → execute

One File, Top to Bottom

All the code in this document is `crud_pdo.php` in `module1/code/2_Connect_PHP_with_MySQL/`.

- Every code block below is a **section of that same file**, shown in the order it runs.
- Variables created in one section (like `$aliceId`) are still available in the sections below it.

Start with One PDO Connection

```
<?php
$dsn = "mysql:host=localhost;dbname=studentdb;charset=utf8mb4";
$pdo = new PDO($dsn, "ase230", "ase230pass", [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
]);
```

These are the local development credentials created in the previous document.

All the operations below use this same `$pdo` object.

What the PDO Connection Code Means

Code	Purpose
<code>\$dsn</code>	MySQL location and database name.
<code>new PDO(...)</code>	Creates the PHP-to-MySQL connection in <code>\$pdo</code> .
<code>"ase230",</code> <code>"ase230pass"</code>	The MySQL application account and password.
<code>ERRMODE_EXCEPTION</code>	Reports database errors as exceptions.
<code>FETCH_ASSOC</code>	Returns results by column name: <code>\$student['name']</code> .

Use `$pdo` to send every CRUD command to MySQL.

Quick Review: The PDO Pattern

```
$sql = "SQL statement with :placeholders";  
$stmt = $pdo->prepare($sql);  
$stmt->execute([  
    ':placeholder' => $value,  
]);
```

- `prepare()` gives the SQL statement to PDO.
- `execute()` supplies values and runs the statement.
- For Read, `fetch()` or `fetchAll()` receives the result.

Look for this pattern in every CRUD example.

CREATE: Insert Two Students

Before we can update or delete a row, we need records to work with. This example creates Alice and Bob, then saves each generated ID.

- Later, we will update Alice by her ID and delete Bob by his ID. Using IDs makes sure each operation affects the intended row.

```
$sql = "INSERT INTO students (name, email, age, major, year)
        VALUES (:name, :email, :age, :major, :year)";
$stmt = $pdo->prepare($sql);

$stmt->execute([
    ':name' => 'Alice Johnson',
    ':email' => 'alice.johnson@example.com',
    ':age' => 22,
    ':major' => 'Computer Science',
    ':year' => 3,
]);
$aliceId = (int) $pdo->lastInsertId();

$stmt->execute([
    ':name' => 'Bob Smith',
    ':email' => 'bob.smith@example.com',
    ':age' => 21,
    ':major' => 'Mathematics',
    ':year' => 2,
]);
$bobId = (int) $pdo->lastInsertId();
```

Save Each New Student's ID

We save Alice's and Bob's IDs so later READ, UPDATE, and DELETE commands affect the correct student, even if the table already contains other rows.

```
$aliceId = (int) $pdo->lastInsertId();
```

- MySQL automatically creates the `id` value when `INSERT` adds a row.
- `lastInsertId()` returns the ID created by the most recent `INSERT`.
- `(int)` converts that returned value to an integer.

READ: Get All Students

`fetchAll()` returns all matching rows — Alice, Bob, and any students from earlier runs (see "Refreshing the Page," later).

(Note: Without an `ORDER BY` clause, SQL does not guarantee row return order.)

```
$sql = "SELECT id, name, email, age, major, year FROM students";
$stmt = $pdo->prepare($sql);
$stmt->execute();
$students = $stmt->fetchAll();

foreach ($students as $student) {
    echo "ID: {$student['id']} - Name: {$student['name']}"
        . " - Major: {$student['major']} - Year: {$student['year']}<br>";
}
```

IMPORTANT

From MySQL Results to PHP Arrays

PHP: build SELECT command

↓ prepare() and execute()

MySQL: find matching rows and return a result set

↓ fetchAll()

PHP: store the rows in the \$students array

↓ foreach

Browser: display each student's values

`execute()` sends the SQL command to MySQL.

`fetchAll()` reads all rows from MySQL's result and converts them into a PHP array, so PHP can use

`$student['name']` and other column values.

Note: This classroom demo uses hardcoded sample data. When values come from user input or untrusted sources, always escape dynamic output using `htmlspecialchars()` before rendering HTML (detailed in Document 4).

Filter One Student with **WHERE**

```
$sql = "SELECT id, name, email, age, major, year  
FROM students  
WHERE id = :id";
```

WHERE filters rows in MySQL: it keeps only rows that meet its condition.

- Here, **:id** is replaced with **\$aliceId**, so only Alice's row is returned.
- Without **WHERE**, the **SELECT** would return every student.

READ: Get One Student (Alice)

Next, use this filter to read Alice's record.

```
$sql = "SELECT id, name, email, age, major, year  
FROM students  
WHERE id = :id";
```

```
$stmt = $pdo->prepare($sql);  
$stmt->execute([':id' => $aliceId]);  
$student = $stmt->fetch();
```

`fetch()` returns one row, or `false` when no row matches.

Use the Result

```
if ($student) {  
    echo "Found {$student['name']} with ID {$student['id']}<br>";  
} else {  
    echo "Student not found<br>";  
}
```

SELECT → prepare → execute → fetch

Read is the only CRUD operation that fetches result rows back from MySQL into PHP memory; Create, Update, and Delete only send a command and check whether it succeeded.

- This is why only Read code calls `fetch()` — the other three have nothing to fetch.

UPDATE: Change Alice

UPDATE changes the matching row. The **WHERE** clause chooses which row; Alice's major and year change; Bob is untouched.

```
$sql = "UPDATE students
        SET major = :major, year = :year
        WHERE id = :id";

$stmt = $pdo->prepare($sql);
$stmt->execute([
    ':major' => 'Computer Engineering', ':year' => 4,
    ':id' => $aliceId,
]);
$updatedRows = $stmt->rowCount();
```

DELETE: Remove Bob

DELETE removes the matching row — here, Bob, not Alice.

```
$sql = "DELETE FROM students WHERE id = :id";  
  
$stmt = $pdo->prepare($sql);  
$stmt->execute([':id' => $bobId]);  
$deletedRows = $stmt->rowCount();
```

Always check the **WHERE** clause before running **UPDATE** or **DELETE**.

Verify Changes: Read Final Results

The first READ showed the students after **INSERT**.

- Now run the same **SELECT** again to verify that Alice was updated and Bob was deleted.

```
$sql = "SELECT id, name, email, age, major, year FROM students";
$stmt = $pdo->prepare($sql);
$stmt->execute();
$students = $stmt->fetchAll();

foreach ($students as $student) {
    echo "ID: {$student['id']} - Name: {$student['name']} "
        . " - Major: {$student['major']} - Year: {$student['year']}<br>";
}
```

What Changed

Step	Alice	Bob
Create	inserted	inserted
Read (all)	found	found
Read (one)	found (filtered by ID)	not queried
Update	major/year changed	unchanged
Delete	—	removed
Final Read	shown, updated	not shown

Update and Delete only make sense once you can see their effect — that's why this document always ends with a final `SELECT`.

- `UPDATE` and `DELETE` do not print any confirmation of their own, so a follow-up `SELECT` is the only way to prove the data actually changed.

Check Changed Rows with `rowCount()`

After `INSERT`, `UPDATE`, or `DELETE`, `rowCount()` reports how many rows MySQL changed:

```
echo "Updated rows: $updatedRows<br>";  
echo "Deleted rows: $deletedRows<br>";
```

This is useful for checking whether a write operation had an effect.

| Why `rowCount()` Can Be `0`

For an `UPDATE`, `0` does not always mean that the row was missing.

- No row matched the `WHERE` condition; or
- A row matched, but the new values were the same as its current values.

Use `SELECT` with `fetch()` when your application must distinguish between these cases. Do not use `rowCount()` to count `SELECT` results.

Run It

In your terminal (from the `ase230` course directory):

```
cd module1/code/2_Connect_PHP_with_MySQL  
php -S localhost:8000
```

Access the server with the web-browser or curl.

```
http://localhost:8000/crud_pdo.php
```

Refreshing the Page (Run PHP Again!)

Each refresh runs the whole PHP file again from the top.

- A **new** Alice and Bob are inserted every time. MySQL's `AUTO_INCREMENT` creates new IDs; `lastInsertId()` returns them to PHP.
- The new Bob is deleted by the end of that same run.
- Alice rows from **previous** runs remain — MySQL data is persistent, so "Get All Students" grows over time. This is expected, not a bug.

To clear the table for a fresh demo, see the optional reset

IMPORTANT

One Mental Model for CRUD

PHP chooses the operation and values



PDO prepares and executes the SQL



MySQL changes or returns persistent data

INSERT
Create

SELECT
Read

UPDATE
Update

DELETE
Delete

CRUD Becomes REST Later

The same data operations will appear again in a REST API.

CRUD	SQL	HTTP method
Create	INSERT	POST
Read	SELECT	GET
Update	UPDATE	PUT
Delete	DELETE	DELETE

REST will add HTTP routing around the CRUD logic you learned today.

*Note: In REST, **PUT** is conventionally used for replacing an entire resource, while **PATCH** is used for partial updates. For this introductory CRUD mapping, **PUT** serves as the primary example.*

Checkpoint

You should now be able to:

1. Explain why MySQL runs as a separate server.
2. Connect PHP to MySQL using `new PDO(...)`.
3. Use `prepare()` and `execute()` for SQL statements.
4. Use `fetch()` and `fetchAll()` for Read.
5. Match CRUD to `INSERT`, `SELECT`, `UPDATE`, and `DELETE`.
6. Explain why students from earlier runs still appear after a refresh.

Troubleshooting steps and essential security concepts (prepared statements and output escaping) are in the next document, along with optional MySQLi reference.